Grievance360

A Smart Grievance Redressal & Analytics Portal
for Educational Institutions

Project Report

System design, module specification, database schema, algorithms, security implementation and quality assurance

Project title	Grievance360
Subtitle	Smart Grievance Redressal & Analytics Portal
Course	MSc-IT, Semester 3
Team size	3 members (Person A / B / C module split)
Technology stack	Python 3, Flask, SQLite / MySQL, Bootstrap 5.3, vanilla JavaScript
Modules	7 functional modules, 8 database tables, 26 routes
Testing	60 automated end-to-end test cases, all passing
Document generated	20 August 2026

Contents

1. Introduction **5**

1.1 Problem statement 5

1.2 Proposed solution 5

1.3 Why the name 5

1.4 Scope of this document 5

2. Objectives and Scope **6**

2.1 Primary objectives 6

2.2 In scope 6

2.3 Out of scope 6

2.4 Assumptions 6

3. Technology Stack **7**

3.1 Components and justification 7

3.2 Database portability design 7

4. System Architecture **9**

4.1 Architectural pattern 9

4.2 Request lifecycle 9

4.3 Template inheritance 10

5. Module Description **11**

5.1 Module M1 - Authentication and Access Control 11

5.2 Module M2 - Complaint Submission 12

5.3 Module M3 - Complaint Tracking and Interaction 13

5.4 Module M4 - Assignment and Auto-Routing 13

5.5 Module M5 - Complaint Processing 14

5.6 Module M6 - Notification and Live Update 15

5.7 Module M7 - Analytics and Reporting 15

6. Database Design **17**

6.1 Entity overview 17

6.2 Data dictionary - complaints 17

6.3 Indexing strategy 18

6.4 Referential integrity 18

6.5 Date storage 18

7. Core Algorithms **19**

7.1 Priority score 19

7.2 SLA breach detection 19

7.3 Auto-routing classifier 20

8. Security Implementation **21**

8.1 Measures applied 21

8.2 Parameterisation under dynamic filtering 21

8.3 Deployment hardening required 22
9. Responsive and Mobile-First Design 23
9.1 Design premise 23
9.2 Rules applied 23
9.3 Verification 23
10. Quality Assurance and Testing 24
10.1 Testing approach 24
10.2 Result summary 24
10.3 Detailed test cases 24
10.4 Notable verifications 26
10.5 Manual verification checklist 27
10.6 Defects found and resolved 27
11. Limitations and Future Scope 28
11.1 Known limitations 28
11.2 Future enhancements 28
12. Conclusion 29
Appendix A - Project Structure 30
Appendix B - Login Credentials 30
Appendix C - Route Reference 31

1. Introduction

1.1 Problem statement

In most educational institutions a grievance is raised on paper, in a register at a departmental office, or verbally to a member of staff. Once submitted, the complaint effectively disappears: the student has no reference number, no way to check progress, and no evidence that the complaint was ever recorded. Administration has no consolidated view either, so recurring problems are never identified, no department is held to a resolution deadline, and the same issue is reported repeatedly without anyone noticing the pattern.

The absence of an audit trail is the central weakness. Without a record of who changed what and when, accountability cannot be established, and without measurable deadlines, no service standard can be enforced.

1.2 Proposed solution

Grievance360 is a web application that digitises the complete grievance lifecycle. A student submits a complaint from a mobile phone and immediately receives a unique ticket code. The system reads the complaint text and suggests the responsible department automatically. An administrator assigns it to a staff member, who works it through a defined sequence of statuses. Every single status change is written to an audit table together with the remark and the identity of the person who made it.

The student watches this progress on a live timeline that refreshes itself without a page reload. When the complaint is resolved the student rates the response, and if the problem returns the complaint can be reopened in one tap. Administration sees the whole picture through an analytics dashboard: complaint distribution, average resolution time per department, a service-level breach report and the single largest recurring issue.

1.3 Why the name

Grievance360 refers to the complete circle a grievance travels: raised, routed, assigned, worked, resolved, rated and, where necessary, reopened. The system closes that loop, and it also gives a 360-degree view of the data to the administration through analytics.

1.4 Scope of this document

This report documents the delivered system. It specifies the seven functional modules, the database design, the two core algorithms, the security measures applied and the quality assurance carried out. All test results reported in Chapter 10 were produced by executing the application, not by inspection.

2. Objectives and Scope

2.1 Primary objectives

1. Provide students with a mobile-first interface for submitting a grievance in under one minute, with an optional file attachment and an option to remain anonymous.
2. Issue a unique, human-readable ticket code for every complaint so that it can be referenced and tracked.
3. Record an immutable audit trail of every status change, capturing the previous status, the new status, an explanatory remark, the user responsible and the timestamp.
4. Route complaints to the correct department automatically by analysing the complaint text, while leaving the final decision with the administrator.
5. Enforce a service-level agreement by attaching a resolution deadline to every complaint and reporting breaches prominently.
6. Rank the pending queue by an objective priority score rather than by submission date alone.
7. Deliver status changes to the user without a page reload.
8. Present administration with analytics covering distribution, average resolution time, breach volume and the top recurring issue, and allow any filtered view to be exported.

2.2 In scope

- Three user roles - student, staff and administrator - with strict role-based access control and per-record ownership checks.
- Self-service student registration; staff and administrator accounts are provisioned by the administrator.
- Password reset using a single-use, time-limited token.
- Complete complaint lifecycle across seven statuses.
- File attachments restricted by type and size.
- Two-way comment thread between the student and the assigned staff.
- Star rating captured after resolution, and a reopen path.
- Notification records with a live unread counter.
- Search, multi-field filtering and pagination over all complaints.
- Excel and PDF export of the currently filtered view.

2.3 Out of scope

- Email and SMS delivery. There is no mail server available in the college laboratory, so the password-reset link is displayed on screen instead of being emailed. The token generation, expiry and single-use logic are identical to a production email flow; only the delivery channel differs.
- Native mobile applications. The web interface is mobile-first and installed browsers are the delivery mechanism.
- Integration with an existing institutional ERP or student information system.
- Multi-institution tenancy. The system is designed for one institution.

2.4 Assumptions

- Students access the system predominantly from mobile phones. The design assumes a 98 percent mobile and 2 percent desktop split, which is why the interface is built phone-first rather than adapted downwards from a desktop layout.
- Each department has at least one member of staff who can act on assigned complaints.
- Complaint volume is at institutional scale - thousands of records rather than millions - which is why polling is an acceptable live-update mechanism.

3. Technology Stack

3.1 Components and justification

Layer	Technology	Reason for selection
Language	Python 3.14	Readable syntax suits a documented academic project; extensive standard library reduces third-party dependencies.
Web framework	Flask 3.1	Minimal and explicit. Every route, query and template is visible in the source rather than generated by framework magic, which matters when the code must be explained during assessment.
Database	SQLite 3 / MySQL 8	MySQL is the target platform and holds the canonical schema. SQLite is the development default so the project runs with no server installation.
Database driver	PyMySQL 1.1	A pure-Python implementation of the MySQL wire protocol. It installs on any Python version without a C compiler, unlike mysqlclient which requires Visual Studio build tools on Windows.
Authentication	Flask-Login 0.6	Provides session management and the user loader; combined with a custom role decorator for authorisation.
Password hashing	Werkzeug 3.1	Salted PBKDF2 hashing through generate_password_hash and check_password_hash.
Templating	Jinja2	Ships with Flask. Auto-escaping is enabled by default, which neutralises cross-site scripting in rendered output.
Front-end framework	Bootstrap 5.3	A mature responsive grid and component set, delivered by CDN. Removes the need to hand-write breakpoint logic.
Client scripting	Vanilla JavaScript (fetch)	No front-end build step, no framework to learn or explain. Three small self-contained files.
Charting	Chart.js 4.4	Responsive canvas charts with a small API surface.
Spreadsheet export	openpyxl 3.1	Generates a genuine .xlsx workbook with styled headers and frozen panes.
Report generation	fpdf2 2.8	Produces this document with explicit control over margins, fonts and table alignment.

3.2 Database portability design

The application must demonstrate MySQL for assessment purposes but also run on any laboratory machine without a database server installed. Rather than maintaining two versions of the code, the project uses a single set of queries and a compatibility shim.

Every SQL statement in the codebase is written in MySQL style using the percent-s placeholder. When the SQLite engine is selected, a four-line cursor wrapper in db.py rewrites those placeholders into the question marks SQLite expects, and converts result rows into dictionaries so that templates behave identically. Database-specific functions such as NOW, DATE_ADD and TIMESTAMPDIFF are never used; all date arithmetic is performed in Python and passed in as parameters.

```
def execute(self, sql, params=()):
    # MySQL uses %s, SQLite uses ?. Rewrite at the driver boundary
    # so that not one query in the project has to change.
    return self._cursor.execute(sql.replace('%s', '?'), params)
```

Design outcome

Switching the entire application from SQLite to MySQL requires editing exactly one line - DB_ENGINE in config.py - and changing zero queries. Because no query is edited, no query can regress during the switch. This is the practical benefit of isolating dialect differences at a single boundary rather than scattering conditionals through the data access code.

4. System Architecture

4.1 Architectural pattern

The application follows a layered architecture. Each layer has one responsibility and depends only on the layer beneath it, so a change in presentation cannot affect business rules and a change of database engine cannot affect either.

Layer	Files	Responsibility
Presentation	templates/, static/	Renders HTML through Jinja2 templates that all extend a single responsive base layout. Client scripts handle live polling only.
Routing / control	app.py	Receives requests, enforces authentication and authorisation, validates input, orchestrates calls to the layers below and selects a template or JSON response.
Business logic	helpers.py	Priority scoring, SLA breach evaluation, auto-routing, ticket code generation, date handling and upload validation. Contains no SQL and no HTTP.
Data access	db.py	Connection management, the placeholder compatibility shim and four query helpers. The only file that knows which engine is in use.
Configuration	config.py	Engine selection, credentials, upload rules, status vocabulary and page size. No logic.
Persistence	database/	Schema definitions for both engines, initialisation and demo data scripts.

4.2 Request lifecycle

The sequence below traces a single authenticated request from the browser to the response, which is the same path every route follows.

1. The browser issues an HTTP request carrying the session cookie.
2. Flask-Login resolves the cookie to a user record through the registered user loader, populating current_user.
3. The login_required decorator confirms that a user is authenticated; an anonymous request is redirected to the login page.
4. The role_required decorator confirms the user holds one of the roles permitted for that route; a mismatch is redirected with an error message rather than a bare HTTP 403.
5. For routes addressing a specific complaint, can_view performs an ownership check so that a user of the correct role still cannot open another person's record.
6. The route validates submitted input and collects error messages.
7. The route calls helpers for any business decision and db helpers for persistence. A connection is opened once per request and closed automatically by the registered teardown handler.
8. Status-changing routes write an audit row and insert notification records for the affected parties.
9. The route renders a Jinja2 template or returns a JSON payload.

4.3 Template inheritance

Every page in the application extends templates/base.html. The base template owns the viewport declaration, the Bootstrap and stylesheet links, the responsive navigation bar, the notification bell, the flash message region, the mobile bottom navigation and the footer. Feature pages supply only a content block.

This single decision is what keeps the interface consistent. Responsiveness is solved once in the parent and inherited by every child, so a new page cannot be accidentally non-responsive, and a layout correction applies everywhere simultaneously. Repeated components such as the complaint card are defined once as Jinja macros in templates/partials/_macros.html and imported where needed, for the same reason.

5. Module Description

The system comprises seven functional modules. Each is described below with its purpose, the functions it provides, its inputs and outputs, and the business rules it enforces.

#	Module	Primary user	Routes
M1	Authentication and Access Control	All	6
M2	Complaint Submission	Student	1
M3	Complaint Tracking and Interaction	Student	4
M4	Assignment and Auto-Routing	Administrator	1
M5	Complaint Processing	Staff	2
M6	Notification and Live Update	All	3
M7	Analytics and Reporting	Administrator	5

5.1 Module M1 - Authentication and Access Control

Purpose

Establishes identity, maintains the session, and guarantees that each user reaches only the areas and records they are entitled to.

Functions provided

Function	Route	Description
Registration	/register	Self-service student sign-up. Validates name length, email format, minimum password length and password confirmation, and rejects an email already present in the users table.
Login	/login	Verifies the submitted password against the stored hash and establishes a session.
Role redirect	/home	Single decision point that sends each role to its own landing page, so no other route needs to repeat that logic.
Logout	/logout	Terminates the session.
Forgot password	/forgot	Generates a 32-byte URL-safe token with a one-hour expiry and stores it against the user record.
Reset password	/reset/<token>	Validates the token and its expiry, then replaces the password hash and clears the token so it cannot be reused.

Business rules enforced

- Passwords are never stored in plain text. They are hashed with Werkzeug on registration and on reset, and only compared as hashes.
- A failed login returns a single generic message regardless of whether the email exists, so the form cannot be used to enumerate registered accounts.
- The forgot-password page returns the same acknowledgement for a registered and an unregistered address, for the same reason.
- A reset token is single-use and expires after one hour.
- A deactivated account cannot log in even with correct credentials.
- The role_required decorator is applied to every non-public route.

Authorisation model

Role	May view	May modify
Student	Only complaints they raised	Own complaints: add comment, submit rating, reopen
Staff	Complaints assigned to them or routed to their department	Assigned complaints: update status with remark, add comment
Administrator	All complaints and all analytics	Assign or reassign any complaint, update any status

> **Two-stage authorisation**
> Role checks alone are insufficient. A student holds the correct role for the tracking page, but must not open another student's ticket. The application therefore performs a second, record-level ownership check through can_view on every route that addresses a specific complaint. Test case 45 in Chapter 10 verifies this directly.

5.2 Module M2 - Complaint Submission

Purpose

Captures a new grievance and establishes every derived value the rest of the system depends on: the ticket code, the resolution deadline and the initial priority score.

Input fields

Field	Type	Validation
Category	Dropdown, required	Must match an existing category row
Subject	Text, required	Minimum 5 characters, maximum 200
Description	Textarea, required	Minimum 15 characters
Attachment	File, optional	Extension must be png, jpg, jpeg, gif, pdf or docx; maximum 5 MB
Anonymous	Checkbox, optional	When set, staff and administrator views display Anonymous in place of the student name

Processing sequence

1. Validate all fields and collect every error before responding, so the user is not corrected one field at a time.
2. Validate and store the attachment if present. The stored filename is prefixed with twelve random hexadecimal characters, so two users uploading a file of the same name cannot overwrite one another.
3. Read sla_days from the selected category and compute the deadline as the submission time plus that many days.
4. Compute the initial priority score from the category weight.
5. Insert the complaint row, then derive the ticket code from the assigned row identifier and write it back, guaranteeing that the code and the record always correspond.
6. Write the first audit row with a null previous status and the new status Pending.
7. Insert a notification for every administrator.

Ticket code format

```
GRV - <four digit year> - <four digit sequence>

Example:  GRV-2026-0031
```

5.3 Module M3 - Complaint Tracking and Interaction

Purpose

Gives the student continuous visibility of their complaint and the means to interact with it after submission.

Functions provided

Function	Route	Description
Dashboard	/student	All complaints raised by the logged-in student, rendered as responsive cards with status badges and SLA state, together with four summary tiles.
Track	/complaint/<code>	Full detail, attachment link, vertical status timeline built from the audit table, comment thread, and the rating or reopen controls where applicable.
Comment	/complaint/<id>/comment	Appends a message to the thread and notifies the other party - never the author.
Feedback	/complaint/<id>/feedback	Records a rating of 1 to 5 with an optional remark.
Reopen	/complaint/<id>/reopen	Returns a completed complaint to active handling.

Conditional interface rules

- The rating form is rendered only when the status is Resolved or Closed, and only for the student who raised the complaint. The server re-checks both conditions on submission, so hiding the form is a convenience rather than the security control.
- A rating may be submitted only once per complaint, enforced by a unique constraint on complaint_id in the feedback table.
- Submitting a rating transitions the complaint to Closed and records that transition in the audit table.
- The reopen control appears only for a complaint in Resolved, Closed or Rejected state.
- Reopening increments reopen_count, clears resolved_at, recalculates the deadline from the reopen moment, and raises the priority score.

5.4 Module M4 - Assignment and Auto-Routing

Purpose

Determines which department is responsible for a complaint and which member of staff will act on it.

Auto-routing mechanism

Each department carries a keyword vocabulary. When the administrator opens the assignment screen, the complaint subject and description are scanned against every vocabulary and each department receives a score. A keyword found in the subject contributes three points; the same keyword found in the description contributes one. The department with the highest total is presented as the suggestion.

Matching uses whole-word boundaries rather than substring matching. Without that constraint

the keyword ac, for air conditioning, would match inside the word academics and route maintenance complaints to the academic office.

When no keyword matches at all, the system deliberately offers no suggestion and asks the administrator to choose. Presenting a confident but arbitrary guess is worse than presenting none, because an administrator working quickly will accept it.

Assignment support

- The staff dropdown displays each member's current open complaint count, so work can be distributed rather than concentrated.
- Selecting a department filters the staff list to that department's members.
- Assignment sets the department, the assignee and the status Assigned, writes an audit row naming the assignee, and notifies both the assigned staff member and the student.

> **Deliberately simple, and defensible**
> This is a rule-based classifier, not machine learning. That is a considered choice: its decisions can be explained line by line, it requires no training data, it cannot degrade unpredictably, and the keyword lists can be tuned by non-programmers as institutional vocabulary changes. For a fixed set of departments and a stable problem domain, it is the appropriate tool.

5.5 Module M5 - Complaint Processing

Purpose

Provides staff with their work queue and the means to advance a complaint through the workflow.

Status vocabulary

Status	Meaning	Set by
Pending	Submitted, not yet assigned	System on submission
Assigned	Allocated to a member of staff	Administrator
In Progress	Actively being worked	Staff
Resolved	Work complete, awaiting confirmation	Staff
Closed	Confirmed complete	Student rating, or staff
Reopened	Returned to handling after resolution	Student
Rejected	Outside scope or not reproducible	Staff

Processing rules

- The staff queue is sorted by live priority score, descending, so the most urgent complaint is always presented first.
- A remark is mandatory on every status change. The remark is displayed to the student on the timeline and forms the permanent record of what was done.
- Every change writes an audit row capturing the previous status, the new status, the remark, the responsible user and the timestamp.
- Transitions to Resolved or Closed set resolved_at, which is the measurement point for the average resolution time analytic.
- The student is notified on every change.

5.6 Module M6 - Notification and Live Update

Purpose

Delivers state changes to the user without requiring a page reload.

Endpoints and clients

Endpoint	Consumer	Interval	Behaviour
/api/complaint/<code>/status	track.js	5 seconds	Returns current status, colour, deadline, breach flag, days remaining and the audit row count.
/api/notifications	notify.js	15 seconds	Returns the ten most recent notifications and the unread count.
/api/notifications/read	notify.js	On demand	Marks all of the user's notifications as read.

Update strategy

The tracking client updates the status badge, deadline and SLA line in place. It does not attempt to rebuild the timeline in JavaScript. Instead it compares the audit row count returned by the API against the count present when the page was rendered; when the server count is higher, the page is reloaded once so that the server rebuilds the timeline.

This avoids maintaining two independent renderings of the same data - one in Jinja and one in JavaScript - which would inevitably diverge. The frequent, cheap update is handled on the client; the rare, complex update is delegated back to the server.

Security of client rendering

Notification text incorporates complaint subjects, which are user input. Before insertion into the document the client assigns the text to a textContent property and reads back the escaped innerHTML, so markup contained in a subject is rendered as literal characters and cannot execute.

5.7 Module M7 - Analytics and Reporting

Purpose

Converts the complaint record into management information, and exports it.

Analytics produced

Analytic	Method	Presentation
Distribution by status	GROUP BY on status	Doughnut chart, colours matched to the status badges used throughout the interface
Distribution by category	LEFT JOIN with GROUP BY	Horizontal bar chart, so long category names remain readable on a narrow screen
Distribution by department	LEFT JOIN with GROUP BY	Vertical bar chart
Monthly trend	Aggregated in Python by year-month	Line chart
Average resolution time	Difference between created_at and resolved_at, averaged per department in Python	Sorted table showing mean days and the number of resolved complaints contributing
SLA breach report	Filtered query on status and deadline	Red table sorted by deadline, oldest breach first

Analytic	Method	Presentation
Top recurring issue	Highest category count	Highlighted banner identifying the largest single source of grievances
Satisfaction	AVG and COUNT over feedback	Summary tiles

> **Why two analytics are computed in Python rather than SQL**
> Average resolution time and the monthly trend both require date arithmetic. MySQL provides TIMESTAMPDIFF and DATE_FORMAT; SQLite provides julianday and strftime. Using either would break portability and force two versions of the query. Performing the arithmetic in Python keeps a single implementation that runs correctly on both engines, and the aggregation is over resolved complaints only, so the volume is modest.

Search, filtering and export

- Filters cover free-text search across ticket code, subject, description and student name, plus status, category, department, a date range and a breached-only switch.
- Results are paginated at twenty per page.
- Excel export produces a genuine .xlsx workbook with a styled header row, sized columns and a frozen header pane.
- PDF export renders a print-optimised A4 landscape report page and uses the browser print dialogue, which keeps the application free of additional rendering dependencies.
- Both exports reuse the same filter-parsing function as the list screen, so the exported rows are exactly the rows on screen. There is no second implementation of the filter logic that could drift.

6. Database Design

6.1 Entity overview

The schema comprises eight tables. Users of all three roles are held in a single table separated by a role column, which keeps authentication uniform and allows a foreign key from a complaint to point at either the student who raised it or the staff member handling it.

#	Table	Purpose	Key relationships
1	departments	Offices that resolve complaints	Referenced by categories, users, complaints
2	categories	Complaint types, each carrying its own SLA and urgency weight	References departments
3	users	Students, staff and administrators	References departments
4	complaints	The central record	References users, categories, departments
5	complaint_status_history	Audit trail of every change	References complaints, users
6	comments	Student and staff conversation	References complaints, users
7	feedback	Star rating, one per complaint	References complaints, users
8	notifications	Feeds the live unread counter	References users, complaints

6.2 Data dictionary - complaints

The complaints table is the centre of the schema and is documented in full below.

Column	Type	Constraint	Description
id	INT	PK, auto	Surrogate key; also the source of the ticket sequence
ticket_code	VARCHAR(20)	UNIQUE, indexed	Human-readable reference, GRV-YYYY-NNNN
user_id	INT	FK users, indexed	The student who raised the complaint
category_id	INT	FK categories, indexed	Determines SLA days and priority weight
department_id	INT	FK departments, nullable	Populated on assignment
assigned_to	INT	FK users, nullable	The staff member responsible
subject	VARCHAR(200)	NOT NULL	Short statement of the problem
description	TEXT	NOT NULL	Full detail
attachment	VARCHAR(255)	Nullable	Randomised filename within static/uploads
is_anonymous	BOOLEAN	Default 0	Suppresses the student name in staff views
status	VARCHAR(20)	Indexed	Current workflow state
priority_score	INT	Default 0	Stored score; recomputed on read for ordering

Column	Type	Constraint	Description
reopen_count	INT	Default 0	Number of times reopened; contributes to priority
created_at	DATETIME	NOT NULL	Submission timestamp
updated_at	DATETIME	NOT NULL	Last modification
deadline	DATETIME	NOT NULL, indexed	created_at plus category SLA days
resolved_at	DATETIME	Nullable	Measurement point for resolution time

6.3 Indexing strategy

Indexes are placed on the columns the application actually filters, joins or sorts on, rather than on every column.

Index	Supports
complaints(ticket_code)	Direct lookup from the tracking URL and the search box
complaints(status)	Status filter and every dashboard count
complaints(category_id)	Category filter and analytics grouping
complaints(user_id)	Student dashboard
complaints(assigned_to)	Staff dashboard and workload counts
complaints(department_id)	Department filter and analytics
complaints(deadline)	SLA breach detection
users(email)	Login and duplicate registration check
complaint_status_history(complaint_id)	Timeline construction on the tracking page
comments(complaint_id)	Comment thread retrieval
notifications(user_id, is_read)	Composite index serving the unread count, which is the most frequently executed query in the system because every logged-in browser issues it every fifteen seconds

6.4 Referential integrity

Foreign keys are declared on every relationship. Under MySQL with InnoDB these are enforced by default; under SQLite the connection issues PRAGMA foreign_keys = ON on open, because SQLite does not enforce declared foreign keys unless explicitly instructed to.

6.5 Date storage

All timestamps are stored in the format YYYY-MM-DD HH:MM:SS. MySQL accepts this directly as a DATETIME and SQLite stores it as TEXT. Because the format is fixed-width and ordered from the most significant unit downwards, lexicographic string comparison produces the same ordering as chronological comparison. Range filters and ORDER BY therefore behave identically on both engines with no conversion.

7. Core Algorithms

7.1 Priority score

Ordering a queue by submission date alone allows a genuinely urgent complaint to sit behind trivial older ones. Ordering purely by category urgency creates the opposite failure: low-priority complaints are never reached. The priority score combines both, plus a penalty for complaints that have already failed once.

```
priority_score = (category.priority_weight  x 10)
               + (days_since_created        x  2)
               + (reopen_count              x  5)
```

Term	Weight	Rationale
Category weight	x 10	The dominant term. A weight of 1 to 5 is assigned per category, so examination issues outrank library issues from the moment of submission.
Age in days	x 2	Grows without limit. This is the term that guarantees no complaint can be starved: after fifteen days a low-weight complaint overtakes a freshly submitted high-weight one.
Reopen count	x 5	A reopened complaint has already consumed the student's patience and one failed resolution attempt, so it is escalated.

Score interpretation

Score range	Label	Displayed as
60 and above	Critical	Red badge
40 to 59	High	Amber badge
25 to 39	Medium	Blue badge
Below 25	Low	Grey badge

> **Stored and recomputed**
> The score is persisted in a column so that it can be indexed, and recomputed on read wherever ordering matters. The reason is that the age term changes every day while the stored value does not. A derived value may be cached, but only if the code is explicit about when the cache is stale - here, the queue always recalculates.

7.2 SLA breach detection

Each category defines sla_days. On submission the deadline is fixed as the creation time plus that many days. A complaint is in breach when it remains open past its deadline.

```
if status NOT IN ('Resolved', 'Closed', 'Rejected')
        and now() > deadline:
    breached = True
```

- Completed complaints are excluded, so a complaint resolved after its deadline is not reported as an ongoing breach.
- Reopening recalculates the deadline from the reopen moment, giving the department a fresh window rather than an immediate breach.

- Breaches surface in three places: the administrator dashboard panel, the analytics breach table, and a red left border on any affected card or row.

7.3 Auto-routing classifier

A weighted keyword classifier maps free text onto a department.

```
for each department:
    score = 0
    for each keyword in that department's vocabulary:
        if whole-word match in subject:      score += 3
        if whole-word match in description:  score += 1

suggestion = department with the highest score
if every score is zero: no suggestion is offered
```

Design decision	Reason
Subject weighted three times higher than description	A user states the actual problem in the subject line and supplies circumstances in the description. The subject is the stronger signal.
Whole-word boundary matching	Substring matching produces false positives. The keyword ac, for air conditioning, would otherwise match inside academics and misroute the complaint.
No suggestion when nothing matches	An arbitrary but confidently presented guess is worse than silence, because a busy administrator will accept it without review.
Suggestion, not automatic assignment	The classifier advises; the administrator decides. This keeps a human accountable for every assignment.

> **Tuning point**
> The keyword vocabularies in helpers.py are the single highest-value customisation in the codebase. Replacing the generic terms with the vocabulary students at a specific institution actually use - local block names, colloquial terms for facilities - raises routing accuracy immediately, with no change to the algorithm.

8. Security Implementation

8.1 Measures applied

Threat	Countermeasure	Where implemented
SQL injection	Every query is parameterised. No user value is ever formatted into a SQL string, including in the dynamic admin filters, where the WHERE clause is assembled from fixed literals and all values travel in a separate parameter list.	db.py, _build_where in app.py
Password disclosure	Salted PBKDF2 hashing on registration and reset; comparison performed on hashes only. Seed accounts are created by a Python script rather than written into the SQL file, so no hash is committed as a literal.	app.py, init_db.py
Cross-site scripting	Jinja2 auto-escaping on all server-rendered output. Client scripts escape text through textContent before insertion. Chart data is delivered inside an inert JSON script block that the browser will not execute.	Templates, notify.js, analytics.html
Broken access control	Role decorator on every protected route, plus a record-level ownership check on every route addressing a specific complaint.	role_required and can_view in app.py
Account enumeration	Login failure and forgot-password responses are identical whether or not the address is registered.	app.py
Malicious file upload	Extension allow-list, 5 MB size cap enforced by the framework, filename sanitisation, and a random prefix that prevents overwriting an existing file.	helpers.save_upload, config.py
Session hijacking	Flask-Login signed session cookies keyed on the application secret.	app.py, config.py
Information disclosure via errors	Custom 403, 404, 413 and 500 handlers return a styled page rather than a stack trace.	app.py

8.2 Parameterisation under dynamic filtering

Dynamic filtering is where injection defects usually appear, because the query text varies with user input. The implementation keeps the two strictly separate: clause fragments are appended from fixed string literals, while the corresponding values are appended to a parallel parameter list. The number of parameters therefore always matches the number of placeholders, and no user-supplied character ever reaches the SQL parser.

```
if filters['status'] in config.STATUSES:
    clauses.append('c.status = %s')      # fixed literal
    params.append(filters['status'])      # value stays separate
```

The status filter additionally validates the submitted value against the known status vocabulary before use, so an unrecognised value is discarded rather than passed through as a parameter.

8.3 Deployment hardening required

The delivered configuration is a development configuration. Before any deployment outside a laboratory the following are mandatory.

- Set debug to False. The Flask debugger exposes an interactive console and must never be reachable in a deployed environment.
- Replace SECRET_KEY with a value generated from a cryptographically secure source and hold it outside version control.
- Serve over HTTPS and set the session cookie to secure and http-only.
- Run behind a production WSGI server rather than the built-in development server.
- Apply rate limiting to the login and password-reset endpoints.

9. Responsive and Mobile-First Design

9.1 Design premise

The user population is assumed to be 98 percent mobile. The interface is therefore authored for a 360-pixel screen first, with desktop refinements added through minimum-width media queries. This is the reverse of the common approach of designing for desktop and reducing afterwards, and it produces a different result: the phone layout is the designed case rather than the degraded one.

9.2 Rules applied

Rule	Implementation	Problem prevented
Viewport declaration	Present in base.html, inherited by every page	Without it a phone renders the page at desktop width and scales it down, making all text unreadable
Minimum 16-pixel input text	Applied to every input, select and textarea	iOS Safari zooms the viewport when focusing a field below 16 pixels, so the page jumps on every tap
Minimum 44-pixel touch targets	Applied to buttons, navigation links and rating stars	Targets below this size are unreliable with a thumb
Tables become cards below 768 pixels	Two renderings of the same data, toggled by a media query	A horizontally scrolling table is technically functional but unusable one-handed
Bottom navigation on phones	Fixed bar with role-aware items and reserved body padding	Top navigation is out of thumb reach on a large phone; without the reserved padding the last control is unreachable
Filters collapse into an accordion	Collapsed below 768 pixels, forced open above	Seven filter controls would otherwise fill an entire phone screen before any result is visible
Charts sized by their container	responsive true, maintainAspectRatio false, fixed-height parent	With aspect ratio disabled and no parent height, the canvas collapses to zero pixels and the chart silently disappears
Horizontal overflow suppressed	Applied at the root, with explicit wrapping on user text	A single long unbroken string would otherwise widen the page and introduce sideways scrolling everywhere

9.3 Verification

The interface was checked at 320, 360, 768, 1024 and 1440 pixels. At every width there is no horizontal page scroll, no truncated text and no overlapping element.

10. Quality Assurance and Testing

10.1 Testing approach

Testing was performed as automated end-to-end verification against a running instance of the application. A test client issues real HTTP requests, maintains real session cookies, submits real form data and inspects the responses returned. No function was tested in isolation with substituted dependencies.

This approach was chosen deliberately. The defects that matter in a system of this kind are integration defects: a decorator omitted from a route, a redirect that loses a session, an ownership check that returns the wrong record, a template variable that is never passed. Unit tests with mocked dependencies do not detect any of these. Driving the assembled application does.

10.2 Result summary

60	60	0	100%
TEST CASES	PASSED	FAILED	PASS RATE

Area	Cases	Passed	Coverage
Public pages	4	4	All unauthenticated routes
Access control	8	8	Anonymous redirection and role enforcement
Authentication	4	4	Login and role-based landing
Complaint submission	5	5	Validation, ticket generation, audit initialisation
JSON API	4	4	Status and notification endpoints
Assignment and routing	8	8	Auto-routing suggestion, workload, assignment
Staff processing	5	5	Status transitions and auditing
Feedback and reopen	5	5	Conditional display, rating, SLA reset
Comments	1	1	Thread insertion
Data isolation	1	1	Cross-user record access
Analytics and export	10	10	All analytics, filters, pagination, both exports
Password reset	3	3	Token, update, re-login
Error handling	2	2	404 and input validation

10.3 Detailed test cases

The table below lists every executed case with its expected result. The actual result matched the expected result in all sixty cases.

#	Test case	Expected result	Status
1	Landing page loads	HTTP 200 with hero and statistics	Pass
2	Login page loads	HTTP 200 with login form	Pass
3	Registration page loads	HTTP 200 with registration form	Pass

#	Test case	Expected result	Status
4	Forgot-password page loads	HTTP 200 with email form	Pass
5	Student area protected	Anonymous request redirected to login	Pass
6	Staff area protected	Anonymous request redirected to login	Pass
7	Admin area protected	Anonymous request redirected to login	Pass
8	Analytics protected	Anonymous request redirected to login	Pass
9	Student login and redirect	Session created, lands on /student	Pass
10	Student dashboard renders	Complaint cards and summary tiles	Pass
11	Admin login and redirect	Lands on /admin	Pass
12	Staff login and redirect	Lands on /staff	Pass
13	Student denied admin area	Redirected with error message	Pass
14	Student denied staff area	Redirected	Pass
15	Student denied analytics	Redirected	Pass
16	Student denied complaint list	Redirected	Pass
17	Submission form loads	HTTP 200 with category dropdown	Pass
18	Categories populated	Six categories, each carrying an SLA	Pass
19	Complaint created	Ticket GRV-2026-0031 generated	Pass
20	Redirect to tracking page	Lands on the ticket URL	Pass
21	First audit row written	Timeline shows submission entry	Pass
22	Status API responds	JSON reports status Pending	Pass
23	API returns deadline	Formatted deadline present	Pass
24	API audit count	history_count equals 1 on a new complaint	Pass
25	Notification API responds	Unread count and item list returned	Pass
26	Priority queue renders	Ordered by descending priority score	Pass
27	Breach panel renders	Overdue complaints listed	Pass
28	Admin search	Query matches the new complaint	Pass
29	Assignment page loads	HTTP 200 with staff dropdown	Pass
30	Auto-routing suggestion	Text containing wifi, printer, lab routed to IT Support	Pass
31	Workload displayed	Four staff, each with open count	Pass
32	Assignment succeeds	Redirect with confirmation	Pass
33	Status becomes Assigned	API confirms Assigned	Pass
34	Staff dashboard loads	Shows only assigned complaints	Pass
35	Assigned ticket visible	New ticket present in staff queue	Pass
36	Status form loads	HTTP 200 with status dropdown	Pass
37	Transition to In Progress	Updated and audited	Pass
38	Transition to Resolved	Updated, resolved_at set	Pass

#	Test case	Expected result	Status
39	Rating form conditional	Appears only once resolved	Pass
40	Rating accepted	Feedback row written	Pass
41	Rating closes complaint	Status becomes Closed	Pass
42	Reopen succeeds	Status becomes Reopened	Pass
43	Reopen resets SLA	New deadline, reopen_count incremented	Pass
44	Comment posted	Message appears in the thread	Pass
45	Cross-user access denied	A second student cannot open another student's ticket	Pass
46	Analytics page loads	HTTP 200	Pass
47	Chart data present	JSON data block rendered	Pass
48	Average resolution table	Days and count shown per department	Pass
49	Top recurring issue	Largest category identified	Pass
50	Breach report present	Overdue complaints tabulated	Pass
51	Status filter	Only matching rows returned	Pass
52	Breached-only filter	Only overdue open rows returned	Pass
53	Pagination	Second page returned correctly	Pass
54	Excel export	Valid xlsx workbook, verified by file signature	Pass
55	PDF export view	Print-optimised report renders	Pass
56	Reset token generated	Single-use link, one-hour expiry	Pass
57	Password updated	Hash replaced, token cleared	Pass
58	Login with new password	Session established	Pass
59	Unknown ticket	Custom 404 page returned	Pass
60	Input validation	Short subject and description rejected	Pass

10.4 Notable verifications

Data isolation - case 45

A complaint was raised by one student, then requested by a second student holding an identical role. Access was refused. This confirms that authorisation is enforced per record and not only per role, which is the defect class most likely to be missed by role-only testing.

Auto-routing correctness - case 30

A complaint whose text contained the terms wifi, printer and lab was submitted, and the classifier suggested IT Support without human input. This verifies the scoring, the subject weighting and the word-boundary matching together.

Export integrity - case 54

The spreadsheet export was verified by examining the returned bytes rather than the HTTP status code. The response begins with the ZIP container signature that identifies a valid Office Open XML workbook and exceeds three kilobytes. A test asserting only a 200 response would pass even if the endpoint returned an error page.

Lifecycle integrity

Cases 19 to 44 execute the complete lifecycle in sequence against a single complaint: submission, automatic routing, assignment, two status transitions, rating, automatic closure, reopening and commenting. Each step reads the state back through the API rather than assuming the previous step succeeded.

10.5 Manual verification checklist

File handling and visual behaviour cannot be meaningfully asserted over HTTP and are verified manually.

#	Check	Expected result
M1	Attach a JPG below 5 MB	Stored with randomised name, link shown on tracking page
M2	Attach an executable file	Rejected with file-type message
M3	Attach a file above 5 MB	Rejected with size message
M4	Submit anonymously	Staff and admin views display Anonymous in place of the name
M5	Live status update	Student tracking page updates within five seconds, no reload
M6	Notification bell	Unread badge appears within fifteen seconds of a change
M7	Duplicate email registration	Rejected with a clear message
M8	Mismatched password confirmation	Rejected before insertion
M9	Interface at 360 pixels	No horizontal scroll; tables render as cards; bottom navigation present
M10	Interface at 1440 pixels	Content column constrained; tables render normally

10.6 Defects found and resolved

No functional defects remained at the conclusion of testing. Two issues were identified and corrected during development.

Issue	Cause	Resolution
Jinja macros rendered without global helper functions	Macros imported into a template do not inherit the template context by default, so functions supplied by the context processor were undefined inside them.	Every macro import declares with context.
Ticket code could not be derived before insertion	The code embeds the row identifier, which is not known until the row exists.	Insert with a placeholder code, then immediately update using the identifier returned by the insert, guaranteeing the code and the row always correspond.

11. Limitations and Future Scope

11.1 Known limitations

Limitation	Current behaviour	Reason
No email delivery	The password reset link is displayed on screen	No mail server is available in the laboratory. Token generation, expiry and single-use enforcement are production-equivalent; only delivery differs.
PDF export uses the browser	A print-optimised page is rendered and the browser print dialogue produces the file	Avoids a heavyweight rendering dependency that would need installing on every machine.
Polling rather than push	Clients poll at five and fifteen second intervals	Appropriate at institutional scale and traverses restrictive networks reliably. Would not scale to very large user counts.
Rule-based routing	Keyword vocabularies require manual maintenance	Explainable and requires no training data, but does not adapt automatically to new terminology.
Single institution	One department and category set	Multi-tenancy was outside the defined scope.

11.2 Future enhancements

1. Email and SMS notification on status change, using the existing notification records as the delivery queue.
2. Server-sent events or WebSockets in place of polling, removing periodic requests entirely.
3. A supervised classifier trained on historical assignments, retaining the keyword classifier as an explainable fallback.
4. Duplicate detection at submission time, identifying complaints about the same underlying problem and grouping them.
5. Escalation rules that reassign or notify a supervisor automatically once a complaint passes its deadline.
6. A staff performance view reporting resolution time and satisfaction rating per individual.
7. Multi-language interface, given that grievances are often described most precisely in a first language.
8. A progressive web app manifest enabling installation to the phone home screen and offline draft composition.

12. Conclusion

Grievance360 delivers a complete, tested grievance management system. Every objective defined in Chapter 2 has been implemented and verified: complaints are submitted from a mobile-first interface, issued a unique reference, routed automatically, assigned with workload visibility, processed through an audited workflow, constrained by a service-level deadline, ranked by an objective priority score, updated live without a page reload, and reported on through analytics with export in two formats.

Three design decisions account for most of the system's durability. Isolating database dialect differences at a single boundary allows the project to run without a database server while remaining a MySQL application, and to switch between the two without editing a query. Placing all responsiveness in one inherited base template makes it impossible for a new page to be non-responsive and allows a layout correction to apply everywhere at once. Separating business logic from routing keeps each rule in one place where it can be located, explained and changed.

The audit trail is the feature that addresses the original problem most directly. Every status change records what changed, who changed it, when, and why. That record is what converts a complaint from a request that may or may not be acted upon into an accountable commitment with a deadline attached.

Testing was conducted end to end against the running application, and all sixty cases pass. The system is complete and functional as delivered.

7	8	26	60
MODULES	TABLES	ROUTES	TESTS PASSED

Appendix A - Project Structure

```
Grievance360/
  app.py                      routes, grouped by role
  config.py                   engine switch, upload rules, statuses
  db.py                       connections, placeholder shim, helpers
  helpers.py                  business logic
  init_db.py                  schema creation and account seeding
  requirements.txt
  database/
    schema.sql                MySQL schema (canonical)
    schema_sqlite.sql         same tables, SQLite syntax
    sample_data.py            30 demo complaints with history
    grievance360.db           created by init_db.py
  docs/
    report_layout.py          PDF layout engine
    generate_report.py        this document
  static/
    css/style.css             all shared layout and responsiveness
    js/track.js               live status polling
    js/notify.js              notification bell polling
    js/charts.js              analytics charts
    uploads/                  complaint attachments
  templates/
    base.html                 the responsive shell
    partials/_macros.html     reusable components
    student/  staff/  admin/  role-specific pages
```

Appendix B - Login Credentials

Role	Email	Password
Administrator	admin@grievance360.edu	admin123
Staff - IT Support	it.staff@grievance360.edu	staff123
Staff - Maintenance	maintenance.staff@grievance360.edu	staff123
Staff - Academics	academics.staff@grievance360.edu	staff123
Staff - Hostel	hostel.staff@grievance360.edu	staff123
Student	harsh@student.edu	student123
Student	priya@student.edu	student123
Student	arjun@student.edu	student123

> **Development credentials only**
> These accounts exist for demonstration and testing. They must be removed or have their passwords changed before any deployment.

Appendix C - Route Reference

Route	Methods	Access	Purpose
/	GET	Public	Landing page
/register	GET, POST	Public	Student registration
/login	GET, POST	Public	Authentication
/logout	GET	Any	End session
/forgot	GET, POST	Public	Generate reset token
/reset/<token>	GET, POST	Public	Set new password
/home	GET	Any	Role-based redirect
/student	GET	Student	Student dashboard
/complaint/new	GET, POST	Student	Submit complaint
/complaint/<code>	GET	Owner, staff, admin	Tracking page
/complaint/<id>/comment	POST	Owner, staff, admin	Add comment
/complaint/<id>/feedback	POST	Owner	Submit rating
/complaint/<id>/reopen	POST	Owner	Reopen complaint
/staff	GET	Staff	Assigned queue
/complaint/<id>/status	GET, POST	Staff, admin	Update status
/admin	GET	Admin	Dashboard, queue, breaches
/admin/complaints	GET	Admin	Search, filter, paginate
/complaint/<id>/assign	GET, POST	Admin	Auto-routing and assignment
/admin/analytics	GET	Admin	Analytics
/admin/export/excel	GET	Admin	Spreadsheet export
/admin/export/pdf	GET	Admin	Printable report
/api/complaint/<code>/status	GET	Owner, staff, admin	Live status
/api/notifications	GET	Any	Notification list
/api/notifications/read	POST	Any	Mark all read